المدرسة الوطنية للعلوم التطبيقية -أكادير
ⵜⴰⵙⴷⴰⵡⵜ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵏ ⵜⵙⵙⵏⴰⵢⵉⵏ ⵜⵉⵜⵜⴰⵡⵉⵏ - ⴰⴳⴰⴷⵉⵔ.
ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES - AGADIR

Documentation du projet

Flight Price Prediction

Réalisé par :

EL IDRISSI Salma
EZZAITOUNI Aya

Encadré par :

Pr. A. QAZDAR

Filière : Finance et Ingénierie Décisionnelle

Année universitaire : 2025–2026 **Déposé le : 11 décembre 2025**

Résumé

Ce projet, réalisé dans le cadre du module de *Data Science*, porte sur l'analyse de données réelles issues de la plateforme de réservation aérienne EaseMyTrip, avec pour objectif de comprendre les facteurs qui influencent le prix des billets d'avion et d'élaborer un modèle prédictif capable d'estimer ces prix. Le dataset étudié contient plus de 300 000 enregistrements et 11 variables décrivant différents aspects d'un vol, tels que la compagnie aérienne, la ville de départ, la classe de voyage, la durée ou encore le nombre de jours restants avant le départ.

Après avoir effectué un nettoyage rigoureux et un prétraitement complet des données, une analyse exploratoire a permis de dégager plusieurs tendances majeures concernant la variation des prix selon les compagnies, les classes, les horaires et la proximité de la date de départ. Plusieurs algorithmes de machine learning — dont la Régression Linéaire, les arbres de décision, Random Forest, Gradient Boosting et XGBoost — ont ensuite été entraînés et comparés selon cinq métriques d'évaluation. Le meilleur modèle obtenu a été sauvegardé puis déployé grâce à une API développée avec FastAPI, et intégré dans une interface web réalisée avec Flask.

Ce travail illustre l'application d'un cycle complet de data science : collecte, préparation, analyse, modélisation et déploiement, montrant ainsi la pertinence des méthodes de science des données pour la prévision des prix dans le secteur aérien.

Table des matières

Résumé **2**

1 Introduction générale **8**

2 Présentation du dataset **10**
2.1 Source et constitution des données 10
2.2 Description des variables 10
2.3 Statistiques descriptives 11
2.3.1 Variables numériques 11
2.3.2 Variables catégorielles 11
2.4 Observations importantes 11

3 Nettoyage et Prétraitement des données **12**
3.1 Introduction 12
3.2 Vérification des valeurs manquantes 12
3.3 Suppression des colonnes inutiles 12
3.4 Encodage des variables catégorielles 13
3.4.1 Encodage de la variable `class` 13
3.4.2 One-Hot Encoding des autres variables catégorielles 13
3.5 Standardisation des variables numériques 13
3.6 Résumé du dataset prétraité 14

4 Analyse exploratoire des données (EDA) **15**
4.1 Prix vs Compagnie aérienne 15
4.2 Prix vs Classe de voyage 16
4.3 Distribution des classes 16
4.4 Prix vs Jours avant le départ 17
4.5 Prix vs Heures de départ 17
4.6 Prix vs Villes de départ et d'arrivée 18
4.7 Distribution globale des prix 18
4.8 Relations entre variables numériques 19
4.9 Heatmap de corrélation 20

4.10 Synthèse des observations 20

5 Sélection des caractéristiques 21
5.1 Importance des variables selon Random Forest 21
5.2 Sélection univariée avec SelectKBest 22
5.3 Comparaison et sélection finale des variables 23
5.4 Conclusion 23

6 Modélisation et évaluation des modèles 24
6.1 Description des modèles utilisés 24
6.1.1 Régression Linéaire 24
6.1.2 Arbre de Décision 24
6.1.3 Random Forest 24
6.1.4 Gradient Boosting 25
6.1.5 K-Nearest Neighbors (KNN) 25
6.1.6 AdaBoost 25
6.1.7 Extra Trees 25
6.2 Entraînement des modèles 25
6.3 Évaluation des modèles 25
6.4 Analyse et choix du meilleur modèle 26
6.5 Conclusion du chapitre 26
6.5.1 Vérification des features avant prédiction 26

7 Déploiement du modèle 27
7.1 Sauvegarde du modèle 27
7.2 Déploiement via FastAPI 27
7.3 Remarque sur le jeu de données 28
7.4 Test des features 28
7.5 Observations 28

8 Développement de l'interface Web avec Flask 30
8.1 Introduction 30
8.2 Architecture du module Flask 30
8.3 Code de l'application Flask 31
8.4 Interface utilisateur HTML 32
8.5 Communication entre Flask et FastAPI 38
8.6 Conclusion 39

9 Conclusion générale 40
9.1 Résumé du projet 40
9.2 Synthèse des principaux enseignements 40

9.3 Réponses synthétiques aux objectifs fixés 41
9.4 Apports du projet 41
9.5 Limites et perspectives d'amélioration 42
9.6 Conclusion finale 42

Table des figures

4.1 Distribution des prix selon les compagnies aériennes 15
4.2 Distribution des prix selon la classe de voyage 16
4.3 Répartition des billets par classe 16
4.4 Prix en fonction du nombre de jours avant le départ 17
4.5 Prix selon la plage horaire de départ 17
4.6 Prix selon la ville de départ 18
4.7 Prix selon la ville d'arrivée 18
4.8 Histogramme des prix des billets 19
4.9 Relations entre variables numériques (pairplot) 19
4.10 Corrélation entre variables numériques 20

5.1 Top 10 des variables les plus importantes selon Random Forest 22

7.1 Liste des features utilisées pour la prédiction avec son output 29

8.1 Illustration de l'interface web permettant de remplir le formulaire. 39
8.2 Résultat de la prédiction affiché après l'envoi du formulaire. 39

Liste des tableaux

2.1 Description des variables du dataset final 10
2.2 Statistiques descriptives des variables numériques 11

3.1 Résumé du dataset après nettoyage et prétraitement 14

6.1 Performance des modèles de régression 26

9.1 Synthèse des observations significatives 41

Chapitre 1

Introduction générale

Les prix des billets d'avion varient fortement en fonction de nombreux facteurs tels que la compagnie aérienne, la classe de voyage, les horaires de départ et d'arrivée, le nombre d'escales ou encore le moment de l'achat. Cette fluctuation permanente rend la planification difficile pour les voyageurs et pose également des défis pour les compagnies aériennes.

Dans le cadre du module de *Data Science*, ce projet a pour objectif de traiter un dataset réel issu de la plateforme EaseMyTrip afin d'analyser les facteurs influençant les prix et de construire un modèle prédictif fiable. Le dataset contient plus de 300 000 observations collectées entre février et mars 2022, avec 11 variables décrivant chaque vol : compagnie aérienne, ville de départ et d'arrivée, classe, durée du vol, jours restants avant le départ, etc.

La méthodologie adoptée suit les étapes classiques d'un projet de data science :

1. **Exploration initiale du dataset** pour comprendre sa structure et les types de variables.
2. **Nettoyage et prétraitement des données**, incluant la gestion des valeurs manquantes, l'encodage des variables catégorielles et éventuellement la normalisation des variables numériques.
3. **Analyse exploratoire des données (EDA)** afin d'identifier les tendances et les relations entre les variables et le prix des billets.
4. **Sélection des variables pertinentes (feature selection)** pour améliorer la performance des modèles prédictifs.
5. **Modélisation** avec plusieurs algorithmes de Machine Learning (Régression Linéaire, Arbres de décision, Random Forest, Gradient Boosting, XGBoost, etc.).
6. **Évaluation** des modèles selon différentes métriques (MAE, MSE, RMSE, R^2, validation croisée).
7. **Déploiement du modèle** via une API FastAPI et création d'une interface web simple avec Flask.

Ce rapport présente en détail chaque étape, depuis l'analyse des données jusqu'au déploiement opérationnel du modèle, et met en évidence les principaux enseignements tirés de ce projet.

Chapitre 2

Présentation du dataset

2.1 Source et constitution des données

Le projet utilise des données de la plateforme **EaseMyTrip** concernant les billets d'avion entre les six principales villes métropolitaines de l'Inde. Les données initiales sont réparties dans deux fichiers CSV distincts :

— `economy.csv` : billets de la classe Economy

— `business.csv` : billets de la classe Business

Ces deux fichiers ont été fusionnés pour créer le dataset final `Clean_Dataset.csv`, contenant **300 153 lignes** et **12 colonnes**. Ce dataset sera utilisé pour toutes les étapes d'analyse, de modélisation et de prédiction.

2.2 Description des variables

Le tableau ci-dessous présente l'ensemble des variables contenues dans le dataset final :

Variable	**Type**	**Description**	**Exemple**
airline	Categorical	Compagnie aérienne	Vistara
flight	Categorical	Code du vol unique	UK-995
source_city	Categorical	Ville de départ	Delhi
departure_time	Categorical	Plage horaire de départ	Morning
stops	Categorical	Nombre d'escales	zero / one
arrival_time	Categorical	Plage horaire d'arrivée	Afternoon
destination_city	Categorical	Ville d'arrivée	Mumbai
class	Categorical	Classe du billet	Economy / Business
duration	Continuous	Durée du vol (heures)	2.25
days_left	Continuous	Nombre de jours entre la réservation et le départ	1
price	Continuous	Prix du billet (INR)	5 953
Unnamed :0	Index	Identifiant automatique	0

TABLE 2.1 – Description des variables du dataset final

2.3 Statistiques descriptives

2.3.1 Variables numériques

Les principales statistiques descriptives des variables numériques sont présentées ci-dessous :

Variable	Count	Mean	Std	Min	Max
duration	300 153	12.22	7.19	0.83	49.83
days_left	300 153	26.00	13.56	1	49
price	300 153	20 889.66	22 697.77	1 105	123 071

TABLE 2.2 – Statistiques descriptives des variables numériques

2.3.2 Variables catégorielles

Pour les variables catégorielles, on observe :

— **airline** : 6 compagnies distinctes, Vistara étant la plus fréquente ($\tilde{1}27\,859$ vols).
— **source_city et destination_city** : 6 villes principales chacune.
— **class** : 2 catégories (Economy et Business), Economy étant majoritaire ($\tilde{2}06\,666$ billets).
— **stops** : majoritairement des vols directs ("zero").
— **departure_time et arrival_time** : 6 plages horaires regroupées.

2.4 Observations importantes

— Le dataset est équilibré entre les différentes classes de billets, mais la classe Economy est légèrement plus représentée.
— Les variables numériques présentent une grande variation, notamment le prix des billets, ce qui justifie une future standardisation.
— Les variables catégorielles nécessiteront un encodage pour être exploitées par les modèles de machine learning.
— Les colonnes `Unnamed:0` et `flight` ne contiennent pas d'information utile pour la prédiction et seront supprimées dans le chapitre suivant.

En résumé, le dataset final est complet, consolidé et exploitable, mais nécessite un prétraitement détaillé avant l'analyse exploratoire et la modélisation. Les étapes de nettoyage et de préparation des données seront présentées dans le **Chapitre 3 : Nettoyage et prétraitement des données**.

Chapitre 3

Nettoyage et Prétraitement des données

3.1 Introduction

Avant de pouvoir construire des modèles prédictifs fiables, il est essentiel de préparer les données de manière rigoureuse. Ce chapitre décrit les différentes étapes de nettoyage et de prétraitement effectuées sur le dataset `Clean_Dataset.csv`. Ces étapes garantissent que les données sont cohérentes, homogènes et adaptées à la modélisation.

3.2 Vérification des valeurs manquantes

La première étape consiste à vérifier l'existence de valeurs manquantes. Le dataset contient 300153 lignes et 12 colonnes. L'inspection montre qu'aucune colonne ne contient de valeurs nulles, ce qui confirme que les données sont complètes et ne nécessitent pas d'imputation.

```
# Exemple de vérification
df.isnull().sum()
```

3.3 Suppression des colonnes inutiles

Certaines colonnes ne sont pas pertinentes pour la prédiction des prix :

— `Unnamed: 0` : index généré automatiquement, inutile pour l'analyse

— `flight` : identifiant unique de vol, non informatif pour le modèle

Ces colonnes ont été supprimées afin de simplifier le dataset et éviter tout bruit dans les modèles.

```
df = df.drop(columns=['flight', 'Unnamed: 0'], errors='ignore')
```

3.4 Encodage des variables catégorielles

3.4.1 Encodage de la variable `class`

La variable `class` (Economy / Business) est transformée en variable binaire :

```
df['class'] = df['class'].map({'Economy': 0, 'Business': 1})
```

3.4.2 One-Hot Encoding des autres variables catégorielles

Les colonnes catégorielles restantes sont :

```
['airline', 'source_city', 'departure_time',
 'stops', 'arrival_time', 'destination_city']
```

Elles ont été encodées avec la technique One-Hot Encoding, en supprimant la première catégorie pour éviter la redondance :

```
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)
```

Après encodage, le dataset contient 31 colonnes. Cette transformation permet aux modèles de machine learning de traiter correctement les variables catégorielles.

3.5 Standardisation des variables numériques

Les variables numériques `duration` et `days_left` ont des échelles différentes. Pour améliorer la performance et la convergence des algorithmes, elles ont été standardisées :

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df_encoded[['duration', 'days_left']] = scaler.fit_transform(
    df_encoded[['duration', 'days_left']]
)
```

Le dataset est maintenant prêt pour l'Analyse Exploratoire des Données (EDA) et la construction des modèles de prédiction.

3.6 Résumé du dataset prétraité

Caractéristique	**Valeur**
Nombre de lignes	300153
Nombre de colonnes après encodage	31
Valeurs manquantes	0
Variables catégorielles encodées	Oui
Variables numériques standardisées	Oui

TABLE 3.1 – Résumé du dataset après nettoyage et prétraitement

Le dataset est maintenant prêt pour l'Analyse Exploratoire des Données (EDA) et la construction des modèles de prédiction.

Chapitre 4

Analyse exploratoire des données (EDA)

Cette section présente une analyse détaillée des différentes variables du dataset et de leur influence potentielle sur le prix des billets d'avion.

4.1 Prix vs Compagnie aérienne

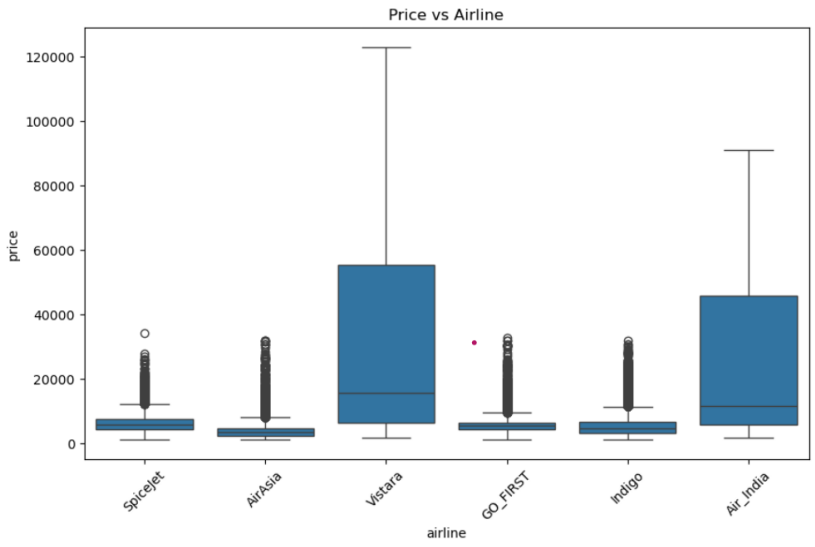


FIGURE 4.1 – Distribution des prix selon les compagnies aériennes

Interprétation : La figure montre que certaines compagnies comme Vistara et Air India ont des prix généralement plus élevés, tandis que GO_FIRST ou SpiceJet proposent des tarifs plus compétitifs. La distribution des prix pour certaines compagnies est très large, indiquant une forte variabilité selon les vols et les destinations. Cela souligne que l'*airline* est un facteur déterminant dans la prédiction des prix.

4.2 Prix vs Classe de voyage

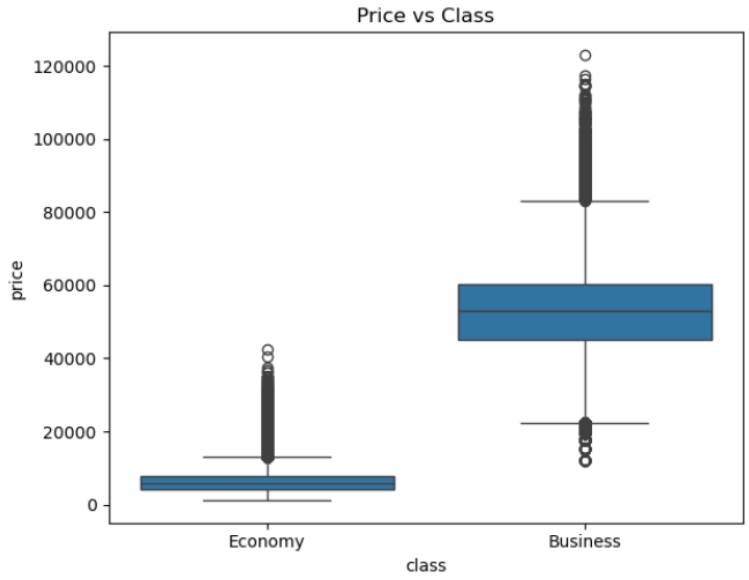


FIGURE 4.2 – Distribution des prix selon la classe de voyage

Interprétation : Les billets Business sont systématiquement beaucoup plus chers que les billets Economy, avec une médiane largement supérieure. La classe est donc un prédicteur très puissant pour le prix.

4.3 Distribution des classes

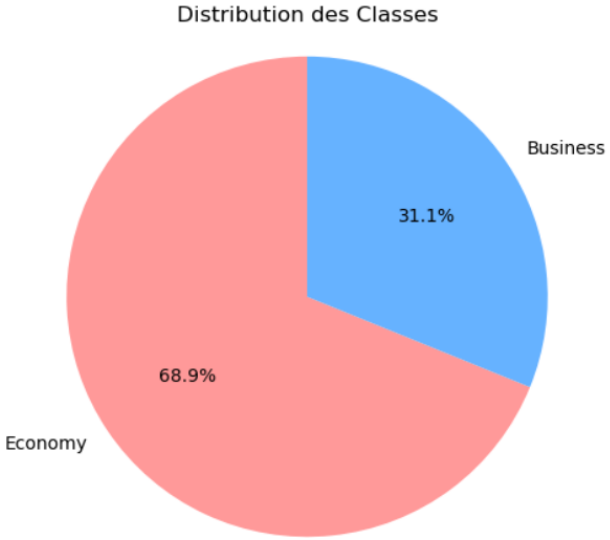


FIGURE 4.3 – Répartition des billets par classe

Interprétation : La majorité des billets sont Economy (69%), ce qui reflète la demande plus élevée pour cette classe. Cela est important à considérer pour équilibrer le modèle lors de la prédiction.

4.4 Prix vs Jours avant le départ

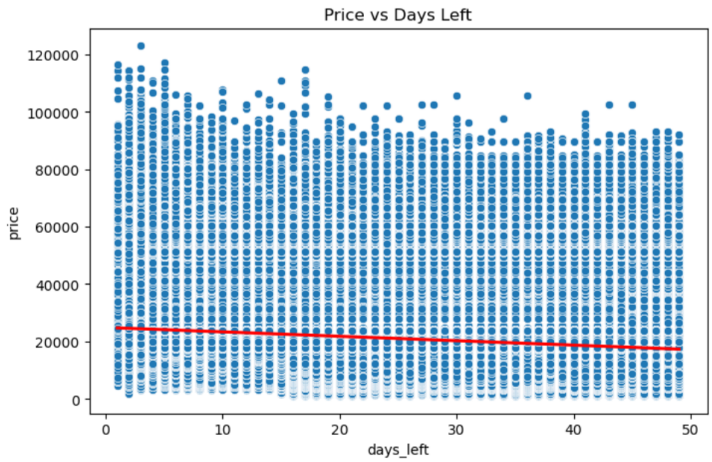


FIGURE 4.4 – Prix en fonction du nombre de jours avant le départ

Interprétation : On observe une légère tendance à la baisse des prix à mesure que la date du vol approche, mais la relation est faible et les prix restent très dispersés. Cela montre que bien que `days_left` influence le prix, d'autres variables doivent être considérées pour une prédiction précise.

4.5 Prix vs Heures de départ

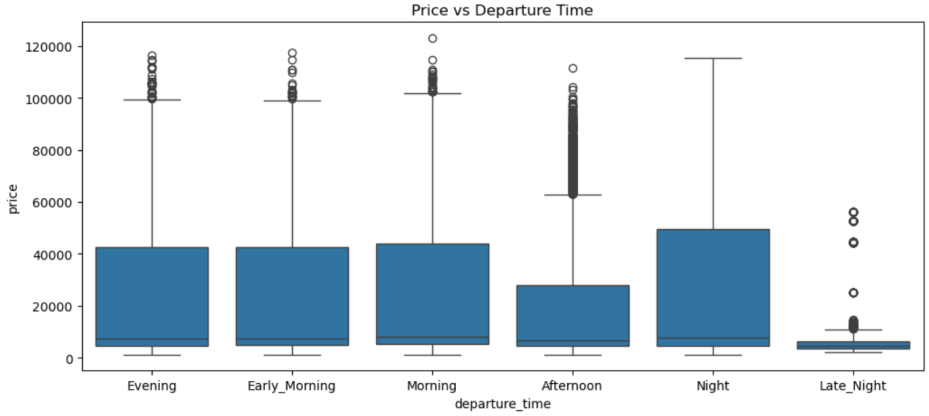


FIGURE 4.5 – Prix selon la plage horaire de départ

Interprétation : Les vols nocturnes ou très tardifs sont généralement moins chers, tandis que les vols aux heures de pointe (matin, après-midi) ont des prix plus élevés. L'heure de départ a donc un impact modéré sur la tarification.

4.6 Prix vs Villes de départ et d'arrivée

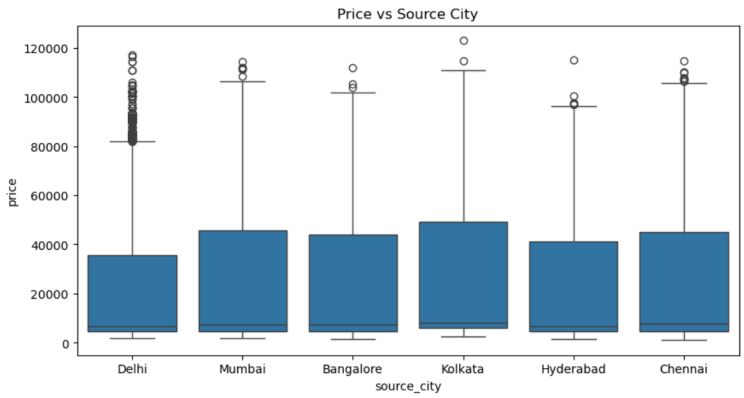


FIGURE 4.6 – Prix selon la ville de départ

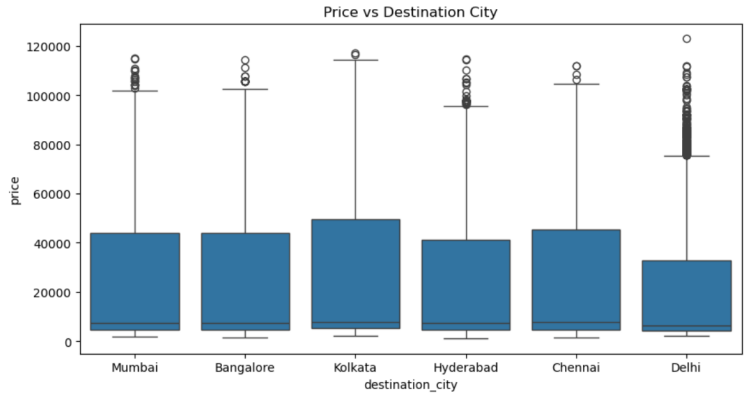


FIGURE 4.7 – Prix selon la ville d'arrivée

Interprétation : Certaines villes comme Delhi ou Mumbai pour le départ et Delhi ou Hyderabad pour l'arrivée affichent des prix moyens plus élevés. Cela montre que la géographie (source et destination) influence fortement le prix, probablement à cause de la demande et de la popularité des trajets.

4.7 Distribution globale des prix

Interprétation : La distribution est très asymétrique avec des prix très élevés pour quelques vols. La majorité des billets se situe sous 20 000 INR, mais il existe des valeurs extrêmes pouvant affecter les modèles de prédiction.

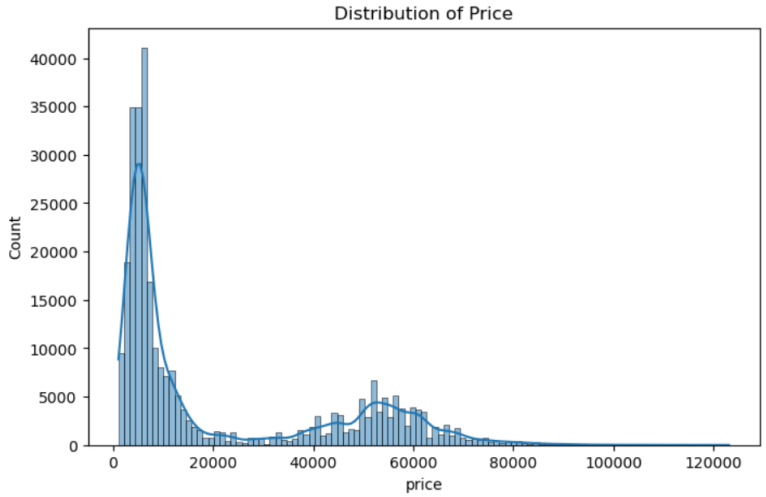


FIGURE 4.8 – Histogramme des prix des billets

4.8 Relations entre variables numériques

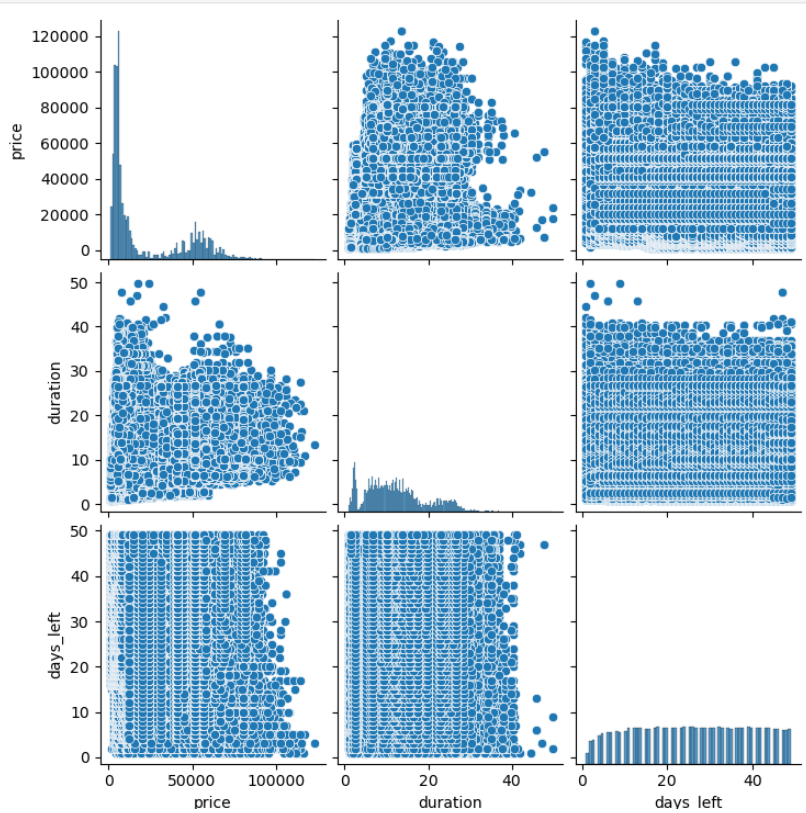


FIGURE 4.9 – Relations entre variables numériques (pairplot)

Interprétation : La durée du vol (`duration`) a une relation positive avec le prix : les vols plus longs coûtent généralement plus cher. Le nombre de jours avant le départ (`days_left`) a un impact plus faible et dispersé sur le prix.

4.9 Heatmap de corrélation

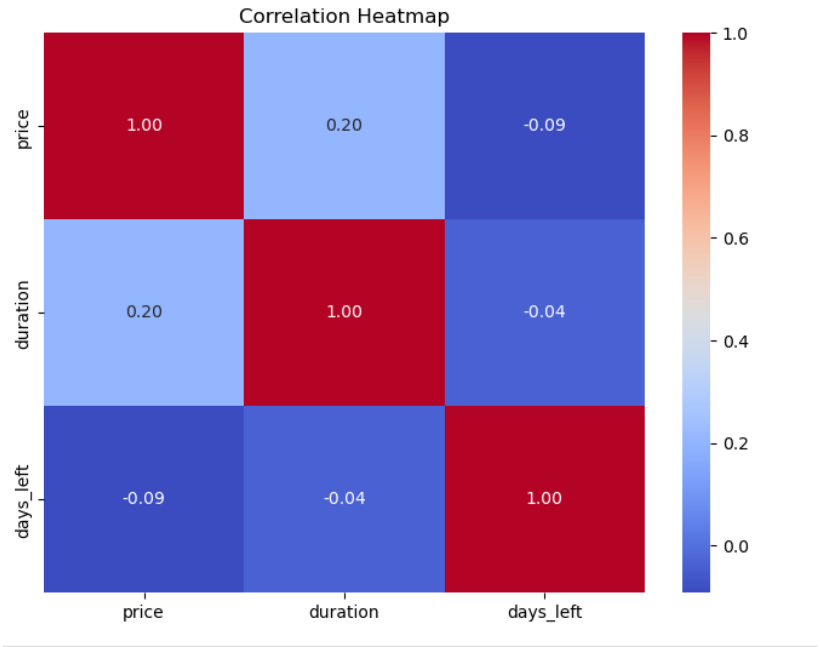


FIGURE 4.10 – Corrélation entre variables numériques

Interprétation : La corrélation entre `price` et `duration` est faible positive (0.20), et entre `price` et `days_left` très faible négative (-0.09). Ces variables numériques seules ne suffisent pas à expliquer complètement le prix, justifiant l'importance des variables catégorielles encodées dans le modèle.

4.10 Synthèse des observations

- La **classe** et la **compagnie aérienne** sont des facteurs déterminants du prix.
- La **ville de départ et d'arrivée** influence fortement les tarifs.
- La **durée du vol** est corrélée positivement avec le prix, alors que le nombre de jours avant le départ a un effet faible.
- L'**heure de départ** a un impact modéré sur le prix.
- La distribution des prix est asymétrique avec quelques valeurs extrêmes.

Conclusion de l'EDA : L'analyse exploratoire montre que le prix des billets est influencé par une combinaison de variables catégorielles et numériques, avec une importance marquée des caractéristiques catégorielles telles que la classe, la compagnie aérienne et les villes de départ/arrivée. Les variables numériques seules (`duration`, `days_left`) ont un effet plus faible. Ces observations guideront la sélection des caractéristiques et la construction des modèles de prédiction dans les chapitres suivants.

Chapitre 5

Sélection des caractéristiques

La sélection des caractéristiques (feature selection) constitue une étape essentielle dans le pipeline de modélisation. Elle permet d'identifier les variables ayant le plus d'influence sur la variable cible `price`, tout en réduisant la complexité du modèle et en améliorant potentiellement les performances de prédiction.

Dans notre cas, la sélection des caractéristiques a été réalisée selon deux approches complémentaires :

1. Analyse de l'importance des variables à l'aide d'un modèle *Random Forest*.
2. Méthode statistique univariée *SelectKBest* basée sur la corrélation linéaire (fonction `f_regression`).

Les résultats obtenus par ces deux méthodes sont présentés ci-dessous.

5.1 Importance des variables selon Random Forest

Le modèle Random Forest constitue un outil efficace pour mesurer l'importance relative des variables, car il capture les relations non linéaires ainsi que les interactions entre caractéristiques.

Après entraînement du modèle sur les données prétraitées, la figure ci-dessous montre les dix variables les plus importantes obtenues :

Observations majeures :

— La variable `class` (Economy/Business) domine très largement l'ensemble des autres caractéristiques, avec une importance de **0.88**. Cela signifie que la classe du billet est de loin le facteur le plus déterminant du prix.

— Les variables numériques `duration` et `days_left` suivent, mais avec des importances beaucoup plus faibles.

— Certaines variables catégorielles encodées (comme `airline_Vistara` ou `airline_Air_India`) apparaissent dans le top 10 mais restent marginales.

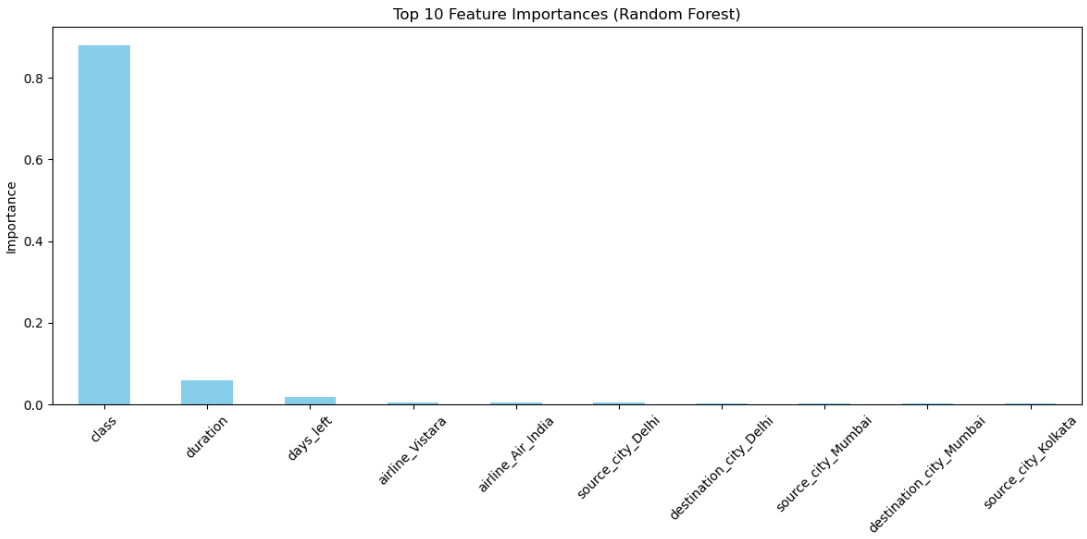


FIGURE 5.1 – Top 10 des variables les plus importantes selon Random Forest

— Les villes de départ et d'arrivée (Delhi, Mumbai, Kolkata) ont un impact modéré mais réel.

Ces résultats confirment que les caractéristiques catégorielles encodées contribuent à la prédiction, mais que leur impact est inférieur à celui de la classe et de la durée du vol.

5.2 Sélection univariée avec SelectKBest

Pour compléter l'approche par modèles ensemblistes, nous avons appliqué la méthode *SelectKBest* avec la fonction `f_regression`, qui sélectionne les variables les plus corrélées statistiquement avec la cible.

Les dix caractéristiques retenues sont :

```
['class', 'duration', 'days_left', 'airline_Air_India',
 'airline_GO_FIRST', 'airline_Indigo', 'airline_SpiceJet',
 'airline_Vistara', 'stops_zero', 'arrival_time_Late_Night']
```

Analyse des résultats :

— La variable `class` ressort encore comme la plus significative, confirmant les résultats du Random Forest.

— Les variables numériques (`duration`, `days_left`) apparaissent logiquement dans la sélection, car elles présentent une corrélation directe avec le prix.

— Les variables liées aux compagnies aériennes (Air India, GO FIRST, Indigo, SpiceJet, Vistara) sont retenues : cela confirme que le choix de l'airline influence le prix.

— Les variables liées aux horaires (`arrival_time_Late_Night`) et aux escales (`stops_zero`) montrent également un rôle non négligeable.

5.3 Comparaison et sélection finale des variables

Les deux méthodes convergent vers les mêmes conclusions :

— La variable `class` est le prédicteur principal du prix du billet.

— Les variables `duration` et `days_left` jouent un rôle complémentaire important.

— Les compagnies aériennes et les villes (source et destination) ont un effet mesuré mais non négligeable.

— Les variables horaires influencent la tarification sans être dominantes.

Au final, toutes les variables conservées dans le dataset `df_encoded` ont été utilisées pour l'entraînement des modèles afin de ne pas exclure d'éventuelles interactions non détectées par les méthodes univariées.

5.4 Conclusion

La sélection des caractéristiques a permis d'identifier les facteurs les plus influents dans la prédiction du prix des billets d'avion. Ces observations guident naturellement la phase suivante, consacrée à l'entraînement et à la comparaison des différents modèles de Machine Learning.

Chapitre 6

Modélisation et évaluation des modèles

L'objectif de cette phase consiste à construire plusieurs modèles de régression afin de prédire le prix des billets d'avion et d'identifier l'algorithme offrant les meilleures performances. Pour assurer une comparaison équitable, l'ensemble des modèles a été évalué sur les mêmes données après un découpage train-test à 80/20.

6.1 Description des modèles utilisés

Afin de couvrir différentes approches d'apprentissage automatique, sept modèles de régression ont été sélectionnés. Chaque modèle est brièvement décrit ci-dessous.

6.1.1 Régression Linéaire

La régression linéaire est un modèle statistique simple supposant une relation linéaire entre les caractéristiques et la variable cible. Elle sert de *baseline* en raison de sa rapidité et de son interprétabilité, bien qu'elle soit limitée pour capturer des relations complexes ou non linéaires.

6.1.2 Arbre de Décision

L'arbre de décision segmente l'espace des données en sous-régions homogènes en apprenant des règles simples sous forme de tests successifs. Il peut modéliser des relations non linéaires, mais il tend à surapprendre si ses paramètres ne sont pas contrôlés.

6.1.3 Random Forest

Random Forest est un ensemble d'arbres de décision construits sur différents sous-échantillons des données. Il améliore la robustesse en réduisant la variance, ce qui en fait un modèle particulièrement performant pour les problèmes de régression complexes. Il fournit également une estimation fiable de l'importance des caractéristiques.

6.1.4 Gradient Boosting

Le Gradient Boosting construit ses arbres séquentiellement, chaque nouvel arbre corrigeant les erreurs du précédent. Ce modèle est efficace pour capturer des interactions complexes, mais peut être sensible au bruit et nécessite un réglage minutieux des hyperparamètres.

6.1.5 K-Nearest Neighbors (KNN)

KNN est un modèle basé sur la similarité : la prédiction est la moyenne des k observations les plus proches. Bien qu'intuitif, il souffre d'une forte complexité computationnelle sur les grands volumes de données et d'une performance réduite lorsque les caractéristiques sont nombreuses.

6.1.6 AdaBoost

AdaBoost est une méthode de *boosting* qui combine plusieurs apprenants faibles (généralement des petits arbres). Chaque modèle successif se concentre sur les observations mal prédites par les précédents. Il offre de bonnes performances mais peut être sensible au bruit dans les données.

6.1.7 Extra Trees

Extra Trees est une variante plus aléatoire de Random Forest où les seuils de séparation sont sélectionnés aléatoirement. Cette propriété permet de réduire encore davantage la variance et d'accélérer l'entraînement. Il est généralement très efficace sur les grands datasets.

6.2 Entraînement des modèles

Les sept modèles ont été entraînés avec les paramètres par défaut, à l'exception de KNN pour lequel un sous-échantillon de 10 000 observations a été utilisé afin de limiter les coûts computationnels. Chaque modèle a ensuite été réentraîné sur l'ensemble complet des données d'entraînement.

6.3 Évaluation des modèles

Les métriques suivantes ont été utilisées :
— **MAE** (Mean Absolute Error),
— **MSE** (Mean Squared Error),

— **RMSE** (Root Mean Squared Error),
— **R^2** score,
— **Cross-Validation** (5-fold).

Modèle	MAE	MSE	RMSE	R^2	CV Score
Linear Regression	4553.29	4.57e7	6761.71	0.9113	-0.2953
Decision Tree	1184.19	1.26e7	3550.00	0.9756	0.3603
Random Forest	**1088.36**	**7.86e6**	**2804.37**	**0.9847**	**0.5672**
Gradient Boosting	2972.06	2.50e7	5001.12	0.9515	0.5135
KNN	6190.55	9.88e7	9938.21	0.8109	-27.3400
AdaBoost	4155.38	3.94e7	6275.77	0.9236	0.4995
Extra Trees	1133.79	8.88e6	2980.50	0.9828	0.5518

TABLE 6.1 – Performance des modèles de régression

6.4 Analyse et choix du meilleur modèle

Les résultats montrent que certains modèles se détachent nettement :
— **Random Forest** obtient les meilleurs résultats sur la quasi-totalité des métriques (MAE, MSE, RMSE, R^2, CV).
— **Extra Trees** et **Decision Tree** offrent aussi de bonnes performances, mais légèrement en dessous.
— La **régression linéaire** est insuffisante pour modéliser la complexité des données.
— **KNN** présente les pires performances, en raison du grand volume de données et de la dimensionnalité élevée.

Le modèle **Random Forest** présente également un faible écart entre performance sur les données d'entraînement et celles de test, ce qui témoigne d'une bonne capacité de généralisation.

6.5 Conclusion du chapitre

Compte tenu de sa précision, de sa stabilité et de sa capacité à capturer les relations non linéaires, **Random Forest est retenu comme modèle final** pour la prédiction du prix des billets d'avion. Il constitue la base du modèle déployé dans le chapitre suivant.

6.5.1 Vérification des features avant prédiction

Avant d'effectuer la prédiction, l'API vérifie que le nombre de features fourni correspond exactement au nombre de features attendu par le modèle. Cela permet d'éviter les erreurs lors de l'inférence et de s'assurer que les données envoyées sont compatibles avec le modèle entraîné.

Chapitre 7

Déploiement du modèle

Après l'entraînement et la sélection du modèle le plus performant (Random Forest), l'étape suivante consiste à le rendre accessible pour des prédictions en temps réel.

7.1 Sauvegarde du modèle

Le modèle est sauvegardé à l'aide de la bibliothèque `pickle` afin de pouvoir être réutilisé sans réentraîner le modèle à chaque fois.

```
import pickle

# Sélection du modèle final
model = models["Random Forest"]

# Sauvegarde du modèle
pickle.dump(model, open('flight_price_model.pkl', 'wb'))
```

7.2 Déploiement via FastAPI

Une API REST est créée pour permettre l'envoi des caractéristiques d'un vol et obtenir en retour la prédiction du prix.

```
# fastapi_app.py
from fastapi import FastAPI, HTTPException
import pickle

# Charger modèle
with open("flight_price_model.pkl", "rb") as f:
    model = pickle.load(f)
```

```
app = FastAPI()

@app.post("/predict")
async def predict(data: dict):
    features = data.get("features")

    if not features:
        raise HTTPException(status_code=400, detail="Missing features")
    if len(features) != 30:
        raise HTTPException(status_code=400, detail="Features must have 30 values")

    try:
        features = [float(x) for x in features]
    except:
        raise HTTPException(status_code=400, detail="All features must be numeric")

    prediction = model.predict([features])[0]
    return {"prediction": float(prediction)}
```

7.3 Remarque sur le jeu de données

Après l'encodage, toutes les colonnes catégorielles ont été transformées en variables booléennes. Ainsi, le jeu de données `X_train` contient principalement des colonnes de type `float` et `bool`, ce qui est compatible avec l'entrée attendue par le modèle Random Forest.

7.4 Test des features

Pour vérifier que les features sont correctement préparées avant la prédiction, une visualisation peut être ajoutée montrant la **liste des features envoyées au modèle** :

7.5 Observations

— Le modèle Random Forest est utilisé pour sa performance et sa stabilité.
— La sauvegarde avec `pickle` permet de réutiliser le modèle pour le déploiement.
— L'API FastAPI rend le modèle accessible via des requêtes HTTP, facilitant son intégration dans une application web ou mobile.

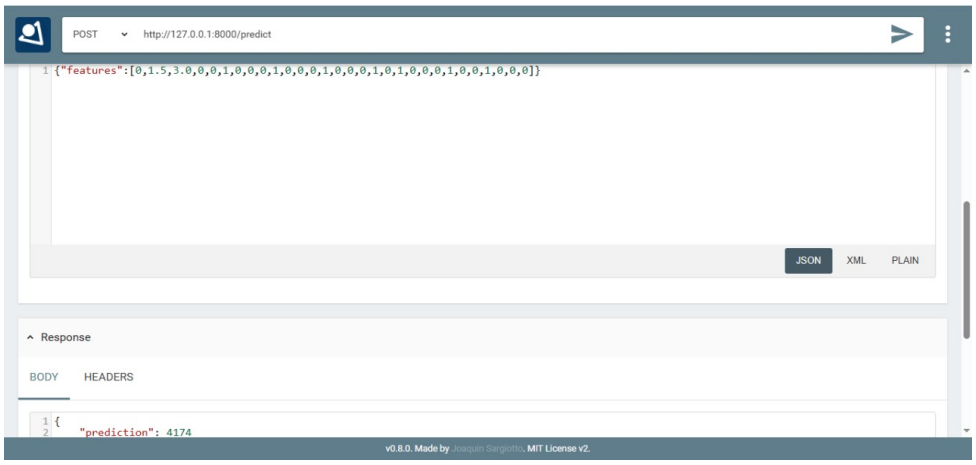


FIGURE 7.1 – Liste des features utilisées pour la prédiction avec son output

— La vérification des features garantit que les entrées respectent le format attendu par le modèle.

Conclusion du déploiement : Cette étape complète le pipeline Data Science en rendant le modèle fonctionnel et utilisable pour la prédiction des prix de billets d'avion à partir de nouvelles données.

Chapitre 8

Développement de l'interface Web avec Flask

8.1 Introduction

Après l'entraînement et le déploiement du modèle via FastAPI, cette étape consiste à développer une interface utilisateur simple permettant d'effectuer une prédiction de prix à partir d'un formulaire web. Pour cela, nous avons utilisé **Flask**, un micro-framework Python permettant de créer des applications web légères. L'interface envoie les données saisies par l'utilisateur au service FastAPI via une requête HTTP POST au format JSON.

8.2 Architecture du module Flask

L'architecture du module Flask est organisée comme suit :

```
project/

 templates/
      index.html

 static/


 app_flask.py
 flight_price_model.pkl  (hébergé côté FastAPI)
```

Le fichier `app_flask.py` contient la logique serveur. Le dossier `templates/` contient la page HTML affichée à l'utilisateur. L'application Flask ne charge pas directement le modèle ; elle interroge FastAPI.

8.3 Code de l'application Flask

Le fichier `app_flask.py` contient le code suivant :

```
from flask import Flask, render_template, request
import requests

app = Flask(__name__)
FASTAPI_URL = "http://127.0.0.1:8000/predict"

@app.route("/", methods=["GET"])
def home():
    return render_template("form.html")

@app.route("/predict", methods=["POST"])
def predict():
    try:
        # Récupérer les 30 features depuis le formulaire
        features = []
        for i in range(30):
            value = request.form.get(f"feature{i}")
            if value is None or value.strip() == "":
                return render_template("form.html", error=f"Feature␣{i+1}␣is␣
                    missing")
            features.append(float(value.strip()))

        # Envoyer àFastAPI
        response = requests.post(FASTAPI_URL, json={"features": features})
        response.raise_for_status()
        result = response.json()

        return render_template("form.html", prediction=result["prediction"])

    except ValueError:
        return render_template("form.html", error="All␣features␣must␣be␣numeric."
            )
    except requests.exceptions.RequestException as e:
        return render_template("form.html", error=f"Server␣error:␣{e}")

if __name__ == "__main__":
    app.run(debug=True)
```

8.4 Interface utilisateur HTML

Le fichier `templates/index.html` contient le formulaire permettant à l'utilisateur de saisir les informations du vol :

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Flight Price Prediction</title>

    <!-- TailwindCSS CDN -->
    <script src="https://cdn.tailwindcss.com"></script>

    <style>
        .fade-in {
            animation: fadeIn 0.6s ease-in-out;
        }
        @keyframes fadeIn {
            from { opacity: 0; transform: translateY(10px); }
            to { opacity: 1; transform: translateY(0); }
        }
    </style>
</head>

<body class="bg-gradient-to-br␣from-blue-50␣to-blue-100␣min-h-screen␣py-10␣px-4"
    >

<h1 class="text-4xl␣font-bold␣text-center␣text-blue-700␣mb-8">
          Flight Price Prediction
</h1>

<div class="max-w-5xl␣mx-auto␣bg-white␣shadow-xl␣rounded-2xl␣p-10␣fade-in">

    <form method="POST" action="/predict">

        <h2 class="text-xl␣font-semibold␣text-blue-800␣mb-6">Enter Flight
            Features</h2>

        <!-- GRID 3 COLUMNS -->
        <div class="grid␣grid-cols-1␣md:grid-cols-3␣gap-6">
```

```
<!-- Example field -->
<div>
    <label class="font-medium␣text-gray-700">Class</label>
    <input type="text" name="feature0"
           class="w-full␣mt-1␣p-3␣border␣rounded-xl␣focus:ring-2␣focus
               :ring-blue-400" required>
</div>

<div>
    <label class="font-medium␣text-gray-700">Duration</label>
    <input type="text" name="feature1"
           class="w-full␣mt-1␣p-3␣border␣rounded-xl␣focus:ring-2␣focus
               :ring-blue-400" required>
</div>

<div>
    <label class="font-medium␣text-gray-700">Days Left</label>
    <input type="text" name="feature2"
           class="w-full␣mt-1␣p-3␣border␣rounded-xl␣focus:ring-2␣focus
               :ring-blue-400" required>
</div>

<!-- Continue all your other 27 fields the same way -->
<!-- I will reproduce all fields for you -->

<div>
    <label class="font-medium␣text-gray-700">Air India</label>
    <input type="text" name="feature3" class="w-full␣mt-1␣p-3␣border␣
        rounded-xl" required>
</div>

<div>
    <label class="font-medium␣text-gray-700">GO_FIRST</label>
    <input type="text" name="feature4" class="w-full␣mt-1␣p-3␣border␣
        rounded-xl" required>
</div>

<div>
    <label class="font-medium␣text-gray-700">Indigo</label>
    <input type="text" name="feature5" class="w-full␣mt-1␣p-3␣border␣
        rounded-xl" required>
```

```
</div>

<div>
    <label class="font-medium␣text-gray-700">SpiceJet</label>
    <input type="text" name="feature6" class="w-full␣mt-1␣p-3␣border␣
        rounded-xl" required>
</div>

<div>
    <label class="font-medium␣text-gray-700">Vistara</label>
    <input type="text" name="feature7" class="w-full␣mt-1␣p-3␣border␣
        rounded-xl" required>
</div>

<div>
    <label class="font-medium␣text-gray-700">Source City  Chennai</
        label>
    <input type="text" name="feature8" class="w-full␣mt-1␣p-3␣border␣
        rounded-xl" required>
</div>

<div>
    <label class="font-medium␣text-gray-700">Source City  Delhi</label
        >
    <input type="text" name="feature9" class="w-full␣mt-1␣p-3␣border␣
        rounded-xl" required>
</div>

<div>
    <label class="font-medium␣text-gray-700">Source City  Hyderabad</
        label>
    <input type="text" name="feature10" class="w-full␣mt-1␣p-3␣border␣
        rounded-xl" required>
</div>

<div>
    <label class="font-medium␣text-gray-700">Source City  Kolkata</
        label>
    <input type="text" name="feature11" class="w-full␣mt-1␣p-3␣border␣
        rounded-xl" required>
</div>
```

```
<div>
    <label class="font-medium␣text-gray-700">Source City  Mumbai</
        label>
    <input type="text" name="feature12" class="w-full␣mt-1␣p-3␣border␣
        rounded-xl" required>
</div>

<div>
    <label class="font-medium␣text-gray-700">Departure  Early Morning<
        /label>
    <input type="text" name="feature13" class="w-full␣mt-1␣p-3␣border␣
        rounded-xl" required>
</div>

<div>
    <label class="font-medium␣text-gray-700">Departure  Evening</label
        >
    <input type="text" name="feature14" class="w-full␣mt-1␣p-3␣border␣
        rounded-xl" required>
</div>

<div>
    <label class="font-medium␣text-gray-700">Departure  Late Night</
        label>
    <input type="text" name="feature15" class="w-full␣mt-1␣p-3␣border␣
        rounded-xl" required>
</div>

<div>
    <label class="font-medium␣text-gray-700">Departure  Morning</label
        >
    <input type="text" name="feature16" class="w-full␣mt-1␣p-3␣border␣
        rounded-xl" required>
</div>

<div>
    <label class="font-medium␣text-gray-700">Departure  Night</label>
    <input type="text" name="feature17" class="w-full␣mt-1␣p-3␣border␣
        rounded-xl" required>
</div>
```

```
<div>
    <label class="font-medium␣text-gray-700">Stops  2 or more</label>
    <input type="text" name="feature18" class="w-full␣mt-1␣p-3␣border␣
        rounded-xl" required>
</div>

<div>
    <label class="font-medium␣text-gray-700">Stops Zero</label>
    <input type="text" name="feature19" class="w-full␣mt-1␣p-3␣border␣
        rounded-xl" required>
</div>

<div>
    <label class="font-medium␣text-gray-700">Arrival  Early Morning</
        label>
    <input type="text" name="feature20" class="w-full␣mt-1␣p-3␣border␣
        rounded-xl" required>
</div>

<div>
    <label class="font-medium␣text-gray-700">Arrival  Evening</label>
    <input type="text" name="feature21" class="w-full␣mt-1␣p-3␣border␣
        rounded-xl" required>
</div>

<div>
    <label class="font-medium␣text-gray-700">Arrival  Late Night</
        label>
    <input type="text" name="feature22" class="w-full␣mt-1␣p-3␣border␣
        rounded-xl" required>
</div>

<div>
    <label class="font-medium␣text-gray-700">Arrival  Morning</label>
    <input type="text" name="feature23" class="w-full␣mt-1␣p-3␣border␣
        rounded-xl" required>
</div>

<div>
    <label class="font-medium␣text-gray-700">Arrival  Night</label>
```

```
            <input type="text" name="feature24" class="w-full␣mt-1␣p-3␣border␣
                rounded-xl" required>
        </div>

        <div>
            <label class="font-medium␣text-gray-700">Destination  Chennai</
                label>
            <input type="text" name="feature25" class="w-full␣mt-1␣p-3␣border␣
                rounded-xl" required>
        </div>

        <div>
            <label class="font-medium␣text-gray-700">Destination  Delhi</label
                >
            <input type="text" name="feature26" class="w-full␣mt-1␣p-3␣border␣
                rounded-xl" required>
        </div>

        <div>
            <label class="font-medium␣text-gray-700">Destination  Hyderabad</
                label>
            <input type="text" name="feature27" class="w-full␣mt-1␣p-3␣border␣
                rounded-xl" required>
        </div>

        <div>
            <label class="font-medium␣text-gray-700">Destination  Kolkata</
                label>
            <input type="text" name="feature28" class="w-full␣mt-1␣p-3␣border␣
                rounded-xl" required>
        </div>

        <div>
            <label class="font-medium␣text-gray-700">Destination  Mumbai</
                label>
            <input type="text" name="feature29" class="w-full␣mt-1␣p-3␣border␣
                rounded-xl" required>
        </div>
    </div>

    <!-- BUTTON -->
```

```
        <button class="mt-8␣w-full␣bg-blue-600␣text-white␣p-4␣rounded-xl␣hover:bg
            -blue-700␣transition-all␣text-lg␣font-semibold␣shadow-lg">
            Predict Price
        </button>
    </form>

    <!-- RESULT -->
    {% if prediction %}
    <div class="mt-8␣p-6␣bg-green-100␣border-l-4␣border-green-500␣text-green-700
        ␣rounded-xl␣fade-in">
        <strong class="text-lg">Predicted Price:</strong> {{ prediction }}
    </div>
    {% endif %}

    <!-- ERROR -->
    {% if error %}
    <div class="mt-8␣p-6␣bg-red-100␣border-l-4␣border-red-500␣text-red-700␣
        rounded-xl␣fade-in">
        <strong class="text-lg">Error:</strong> {{ error }}
    </div>
    {% endif %}

</div>

</body>
</html>
```

8.5 Communication entre Flask et FastAPI

La communication entre les deux services suit le flux suivant :

— l'utilisateur remplit le formulaire HTML ;

— Flask récupère les données et les formate en JSON ;

— Flask envoie une requête POST à FastAPI ;

— FastAPI applique le modèle de Machine Learning et renvoie le prix prédit ;

— Flask affiche le résultat sur la page web.

Ce découplage permet une architecture plus propre et facilite le déploiement,comme illustré ci-dessous.

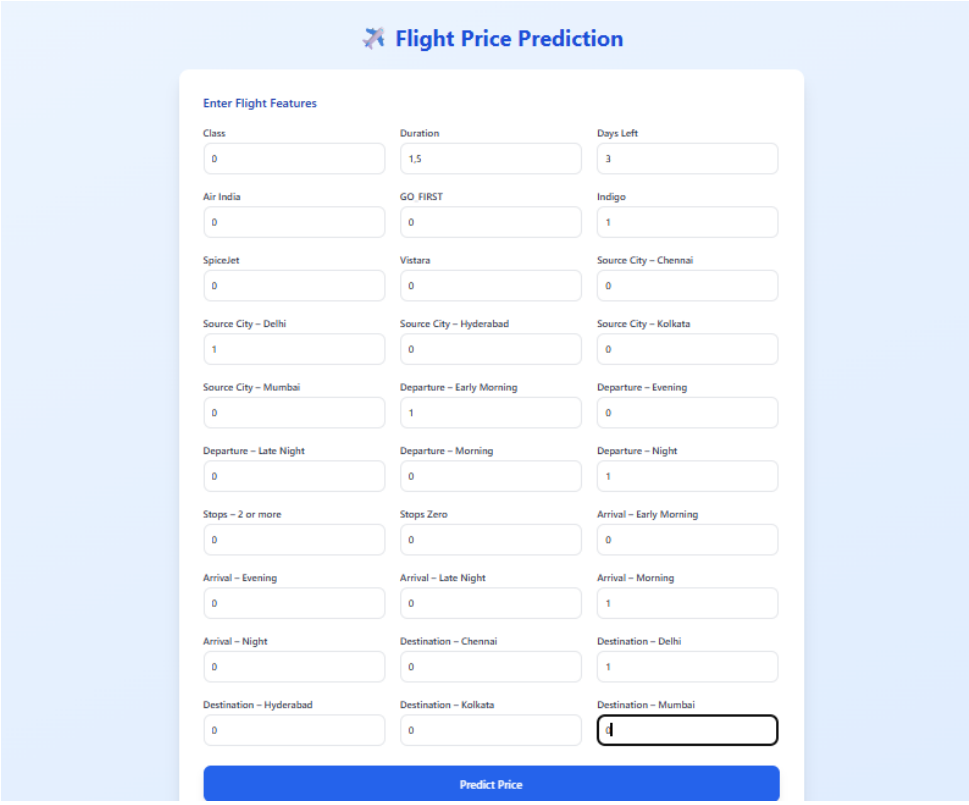


FIGURE 8.1 – Illustration de l'interface web permettant de remplir le formulaire.

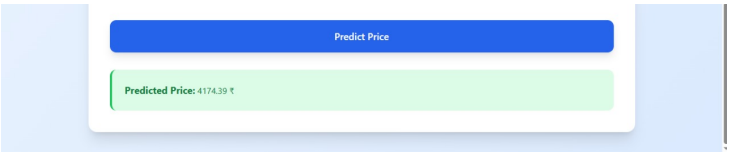


FIGURE 8.2 – Résultat de la prédiction affiché après l'envoi du formulaire.

8.6 Conclusion

L'intégration d'une interface web avec Flask permet d'exposer le modèle de prédiction de manière conviviale et accessible à tout utilisateur, sans manipulations techniques. Elle marque la dernière étape du pipeline complet allant de la préparation des données à l'utilisation réelle du modèle en production.

Chapitre 9

Conclusion générale

9.1 Résumé du projet

L'objectif de ce projet consistait à analyser un ensemble de données de billets d'avion provenant de la plateforme EaseMyTrip et à construire un modèle prédictif permettant d'estimer le prix des billets. La démarche adoptée a couvert l'ensemble du workflow data science, de l'exploration des données à leur modélisation, suivie du déploiement du modèle via une API et d'une interface web.

Le travail s'est articulé autour des étapes suivantes :

— Exploration et compréhension du jeu de données ;
— Nettoyage et prétraitement des variables ;
— Analyse exploratoire pour mettre en évidence les relations entre les variables et le prix ;
— Sélection des caractéristiques pertinentes ;
— Construction et évaluation de différents modèles de régression ;
— Déploiement du modèle retenu à l'aide de FastAPI et développement d'une interface web avec Flask.

9.2 Synthèse des principaux enseignements

L'analyse exploratoire des données a permis de dégager plusieurs conclusions importantes sur les facteurs influençant le prix des billets :

— La classe de voyage (*Economy* vs *Business*) constitue un des facteurs les plus déterminants du prix, les billets Business étant significativement plus chers.
— La compagnie aérienne impacte fortement le tarif moyen : certaines compagnies ont tendance à proposer des prix plus élevés ou plus compétitifs.
— La géographie du vol (ville de départ et d'arrivée) influence également les prix, notamment pour les routes impliquant des hubs importants.

— La durée du vol est corrélée positivement avec le prix, alors que l'effet des jours restants avant le départ est plus faible et dispersé.
— L'heure de départ présente un impact modéré sur les prix, avec des schémas visibles en fonction des plages horaires.

Ces observations ont servi de base pour guider la sélection des variables et orienter la construction des modèles de prédiction.

—

9.3 Réponses synthétiques aux objectifs fixés

Pour répondre aux objectifs définis dans le cahier des charges, une synthèse des résultats principaux est présentée ci-dessous :

TABLE 9.1 – Synthèse des observations significatives

Analyse thématique	**Conclusions principales**
Impact de la compagnie aérienne sur le prix	Les tarifs moyens varient selon les compagnies, indiquant que l'airline est un facteur déterminant.
Effet de la proximité de la date de départ	Une légère diminution des prix est observable à mesure que le départ approche, mais la relation reste faible et dispersée.
Influence des plages horaires	Les vols aux heures de pointe affichent des prix plus élevés, alors que les vols tardifs ou très matinaux tendent à être moins coûteux.
Effet des villes de départ et d'arrivée	Certaines routes (ex. Delhi, Mumbai, Hyderabad) présentent des tarifs moyens plus élevés, montrant l'effet géographique.
Différence entre classes de voyage	Les billets Business sont systématiquement plus onéreux que les billets Economy.

La synthèse ci-dessus montre que les principaux facteurs identifiés influencent de manière significative le prix des billets. Cette analyse guide également la compréhension des comportements observés dans le modèle final.

9.4 Apports du projet

Ce projet a permis de mettre en œuvre une démarche complète de data science appliquée à un problème réel :

— Il fournit une compréhension approfondie des variables influençant les prix des billets.
— Il identifie un modèle robuste et performant pour la prédiction des prix.
— Il montre comment déployer ce modèle dans une application web accessible.
— Il offre une base solide pour des développements ultérieurs dans un contexte réel.

9.5 Limites et perspectives d'amélioration

Bien que le modèle et l'analyse aient donné des résultats pertinents, certaines limites peuvent être mentionnées :

— Le jeu de données couvre uniquement une période et six villes, limitant potentiellement la généralisation du modèle.
— L'exclusion de certaines variables contextuelles (ex. saison, jours fériés, conditions météo) peut réduire la précision dans certains cas.
— L'interface web reste basique et pourrait être enrichie pour une meilleure expérience utilisateur.
— L'interface peut être améliorée et sophistiquée lors de sa création, tout en restant claire, ergonomique et performante

Des perspectives d'amélioration incluent l'intégration de nouvelles sources de données, l'optimisation des hyperparamètres, ou encore l'utilisation de modèles avancés (LightGBM, CatBoost, etc.).

9.6 Conclusion finale

En conclusion, ce projet illustre une application complète de l'analyse de données et du machine learning pour la prédiction des prix de billets d'avion. Les objectifs initiaux ont été atteints, avec des réponses claires et cohérentes aux enjeux fixés dans le cahier des charges. Le modèle développé, associé à son API et son interface web, constitue une solution fonctionnelle prête à être utilisée ou étendue dans des contextes réels.